

Instruction Processor	
NOP	No Operation
CLRL	Clear extended Load Register to zeroes.
MACX,L	Execute Load Register as a Macro
PVMAC	Switch macro specified in Macro address register to Private
GPMAC	Switch macro specified in Macro address register to Group
WREG,DP	Write to External Hex Display
RREG,L	Read from Load Register
WREG,L	Write to Load Register
END	Immediately end this instruction cycle.
LDW***	Shift 16-bit word ##### into Load Register
LDB**	Shift byte ## into Load Register
WRMAC**	Write Load Register into Macro slot ##
MACX,0-15	Execute selected Macro 0 to 15

Register Bank	
RREG,0-9	Read from Group or Private Register 0-9
WREG,0-9	Write to Group or Private Register 0-9
RREG,A	Read from Group Register A
WREG,A	Write to Group Register A
PVREG	Switch last read Register 0-9 to Private
GPREG	Switch last read Register 0-9 to Group

Program Counter Control & Memory	
JMP***	Perform PC Jump to Memory Address Register location
JSR***	Perform PC Subroutine Jump to Memory Address Register location
RET	Return from Subroutine to stored JSR location (Small stack, 4 levels of recursion avail.)
JRST	Clear JSR Stack for this process
LOOP	Record a loop start location (Small stack, 4 levels of recursion avail.)
LRET	Return to loop start location for next process instruction cycle.
LBRK	End loop and pull loop address off of loop stack. Execution continues at next inst. word.
LRST	Clear Loop Stack for this process
REDO	For this process, the next instruction cycle will repeat at this current location.
MREAD	Read from Memory location specified by Memory Address Register
MWRITE	Write to Memory location specified by Memory Address Register
RSTMA	Reset Memory Address Register to initial address location. (PC or Interface addr.)
HOP,RF	Move Memory Address Register Relative Forward
HOP,RR	Move Memory Address Register Relative Right
HOP,RL	Move Memory Address Register Relative Left
HOP,RB	Move Memory Address Register Relative Reverse
HOP,AF	Move Memory Address Register to Absolute Forward Direction
HOP,AR	Move Memory Address Register to Absolute Right Direction
HOP,AL	Move Memory Address Register to Absolute Left Direction
HOP,AB	Move Memory Address Register to Absolute Rear Direction
RREG,MA	Read from full Memory Address Register
WREG,MA	Write to full Memory Address Register
RREG,MX	Read from Memory Address Register, X field
WREG,MX	Write to Memory Address Register, X field
RREG,MY	Read from Memory Address Register, Y field
WREG,MY	Write to Memory Address Register, Y field
RREG,MP	Read from Memory Address Register, Page/Bank field
WREG,MP	Write to Memory Address Register, Page/Bank field
PAGE**	Select Address Page
PDEF**	Define Address Page

Arithmetic Logic Unit	
WREG,UI	Write to Unary ALU Input Register
WREG,B1	Write to Binary ALU Input 1 Register
WREG,B2	Write to Binary ALU Input 2 Register
WREG,AC	Write to ALU Configuration Register
RREG,AC	Read from ALU Configuration Register
RREG,UO	Read from Unary ALU Output Register
RREG,BO	Read from Binary ALU Output Register
RREG,Z	Read zero from static Zero Register
ACOFF	Temporarily disable customized ALU Configuration and use default config.
ACON	Re-enable customized ALU Configuration
ACLR	Clear all ALU Input/Output Registers (but not ALU config register)
**	
LSET**	Set ALU I/O Looping Mode
PASS	Pass Unary Input to Output without change
NOT	Perform Logical Not Operation on Unary ALU
INC	Perform Arithmetic Increment Operation on Unary ALU
DEC	Perform Arithmetic Decrement Operation on Unary ALU
NEG	Perform Arithmetic Negate Operation on Unary ALU
WSW	Perform Upper/Lower Word Swap on Unary ALU
BRR	Perform Byte Rotate Right Operation on Unary ALU
BRL	Perform Byte Rotate Left Operation on Unary ALU
BSET	Perform Bit Set operation on Binary ALU Input 1 with bit position in Input 2
BCLR	Perform Bit Clear operation on Binary ALU Input 1 with bit position in Input 2
BINV	Perform Bit Invert operation on Binary ALU Input 1 with bit position in Input 2
BTST	Perform Bit Test operation on Binary ALU Input 1 with bit position in Input 2
ADD	Perform Arithmetic ADD operation on Binary ALU
ADD,W****	Immediate ADD 2-byte unsig. word to Binary ALU Input 1 and write back to ALU Input 1
ADD,B1**	Immediate ADD unsigned byte to Binary ALU Input 1 and write back to ALU Input 1
ADD,B2**	Immediate ADD unsigned byte to Binary ALU Input 2 and write back to ALU Input 2
ROR	Perform Bit Rotate Right Operation on Unary ALU
ROL	Perform Bit Rotate Left Operation on Unary ALU
RRX	Perform Bit Rotate Right w/Carry Operation on Unary ALU
RLX	Perform Bit Rotate Left w/Carry Operation on Unary ALU
LSR	Perform Bit Logical Shift Right Operation on Unary ALU
LSL	Perform Bit Logical Shift Left Operation on Unary ALU
ASR	Perform Arithmetic Shift Right Operation on Unary ALU
BSL	Perform Byte Shift Left Operation on Unary ALU
OR	Perform Logical OR operation on Binary ALU
XOR	Perform Logical XOR operation on Binary ALU
IMP	Perform Logical Imply operation on Binary ALU
CMP	Perform Arithmetic Compare operation on Binary ALU
SUB	Perform Arithmetic Subtract operation on Binary ALU
SUB,W***	Immediate Subtract 2-byte u.word from Binary ALU Input 1 and write back to ALU Input 1
SUB,B1**	Immediate unsigned Subtract byte from Binary ALU Input 1 and write back to ALU Input 1
SUB,B2**	Immediate unsigned Subtract byte from Binary ALU Input 2 and write back to ALU Input 2
IB0	Isolate Byte 0 from Unary ALU Input and output to Byte 0 of Unary ALU Output
IB1	Isolate Byte 1 from Unary ALU Input and output to Byte 0 of Unary ALU Output
IB2	Isolate Byte 2 from Unary ALU Input and output to Byte 0 of Unary ALU Output
IB3	Isolate Byte 3 from Unary ALU Input and output to Byte 0 of Unary ALU Output
ZB0	Clear Byte 0 on Unary ALU Input and output to Unary ALU Output
ZB1	Clear Byte 1 on Unary ALU Input and output to Unary ALU Output
ZB2	Clear Byte 2 on Unary ALU Input and output to Unary ALU Output
ZB3	Clear Byte 3 on Unary ALU Input and output to Unary ALU Output
NOR	Perform Logical NOR operation on Binary ALU
XNOR	Perform Logical XNOR operation on Binary ALU
INH	Perform Logical Inhibit operation on Binary ALU
NAND	Perform Logical NAND operation on Binary ALU
AND	Perform Logical AND operation on Binary ALU
AND,W****	Immediate AND 2-byte unsig. word with Binary ALU Input 1 and write back to ALU Input 1
AND,B1**	Immediate AND unsigned byte with Binary ALU Input 1 and write back to ALU Input 1
AND,B2**	Immediate AND unsigned byte with Binary ALU Input 2 and write back to ALU Input 2

Context Switching	
FORK,RF	Fork Process with PCD Relative Forward Direction at current PC location
FORK,RR	Fork Process with PCD Relative Right Rotation at current PC location
FORK,RL	Fork Process with PCD Relative Left Rotation at current PC location
FORK,RB	Fork Process with PCD Relative Reversed Direction at current PC location
FORK,AF	Fork Process with PCD Absolute Forward Direction at current PC location
FORK,AR	Fork Process with PCD Absolute Right Direction at current PC location
FORK,AL	Fork Process with PCD Absolute Left Direction at current PC location
FORK,AB	Fork Process with PCD Absolute Rear Direction at current PC location
FORK,MA	Fork Process with PCD Relative Forward at Memory Address Location
TERM	Terminate Process Immediately.
DUPE	Duplicate Process Queue Scheduling Entry and Increment Dupe Counter for Process
TRMD	Terminate Duplicate Process Queue Scheduling Entry and Decrement Dupe Counter
RREG,AP	Read from Active/Dupe Process Counter Register
RREG,EP	Read from Extended Process Info Register
HALT	Stop process execution on CPU
RESET	Stop process execution and reinitialize CPU

Conditional & Flag Subsystem	
CLRA	Clear Flag 'A'
SETA	Set Flag 'A'
CLRB	Clear Flag 'B'
SETB	Set Flag 'B'
CLR_X	Clear Flag 'X'
SET_X	Set Flag 'X'
CLRF**	Clear Specified Flag
SETF**	Set Specified Flag
RREG,FS0	Read Flag State Register 0 with Flags 0-31
RREG,FS1	Read Flag State Register 1 with Flags 32-63
PRVF	Make selected flag private (A,B,X, and G0-GF only)
GRPF	Make selected flag group access (A,B,X, and G0-GF only)
CLRC	Clear Carry Flag
SETC	Set Carry Flag
FSEL**	Select Specified Flag
COND**	Specify Flag for Conditional Operation (use second byte for flag numeric ID)
COND,□	Specify Flag for conditional operation (specify flag character or numeric ID)

Conditional Byte 3/7	
ROT,□	Unconditionally rotate PCD
□,RR	Relative Right
□,RL	Relative Left
□,RB	Relative Back
□,AF	Absolute Forward
□,AR	Absolute Right
□,AL	Absolute Left
□,AB	Absolute Back
CST,□	Use selected flag, compare with true, run action if success
CSF,□	Use selected flag, compare with false, run action if success
CAT,□	Use auto-selected flag, compare with true, run action if success
CAF,□	Use auto-selected flag, compare with false, run action if success
□,NOP	No operation
□,ROT,RR	Rotate Relative Right
□,ROT,RL	Rotate Relative Left
□,ROT,RB	Rotate Relative Back (reverse)
□,ROT,AF	Rotate Absolute Forward
□,ROT,AR	Rotate Absolute Right
□,ROT,AL	Rotate Absolute Left
□,ROT,AB	Rotate Absolute Back
□,REDO	REDO opcode
□,TERM	TERM opcode
□,LRET	LRET opcode
□,LBRK	LBRK opcode
□,RET	RET opcode

** Two-byte opcode: Second byte is data. *** Three-byte opcode: Second and third bytes are data.

	0x0_	0x1_	0x2_	0x3_	0x4_	0x5_	0x6_	0x7_	0x8_	0x9_	0xA_	0xB_	0xC_	0xD_	0xE_	0xF_	Byte 7 opcodes
0x_0	NOP	MACX,0	RREG,0	WREG,0	LOOP	HOP,RF	COND,T	COND,0	WREG,UI	PASS	ROR	IB0	FORK,RF		THR,N,OFF	RREG,IFC	CST,□
0x_1	CLRL	MACX,1	RREG,1	WREG,1	LRET	HOP,RR	COND,F	COND,K	WREG,B1	NOT	ROL	IB1	FORK,RR		THR,N,ON	WREG,IFC	CSF,□
0x_2	MACX,L	MACX,2	RREG,2	WREG,2	LBRK	HOP,RL	COND,Z	COND,X	WREG,B2	INC	RRX	IB2	FORK,RL		THR,E,OFF	RREG,RD	CAT,□
0x_3	PVMAC	MACX,3	RREG,3	WREG,3	LRST	HOP,RB	COND,N	COND,R	WREG,AC	DEC	RLX	IB3	FORK,RB		THR,E,ON	WREG,RD	CAF,□
0x_4	GPMAC	MACX,4	RREG,4	WREG,4	RET	HOP,AF	COND,V	COND,A	RREG,AC	NEG	LSR	ZB0	FORK,AF		THR,S,OFF	DMODE,C	ROT,□
0x_5	WREG,DP	MACX,5	RREG,5	WREG,5	JRST	HOP,AR	COND,C	COND,B	RREG,U0	WSW	LSL	ZB1	FORK,AR		THR,S,ON	DMODE,R	
0x_6	RREG,L	MACX,6	RREG,6	WREG,6	MREAD	HOP,AL	COND,M	COND,P	RREG,B0	BRR	ASR	ZB2	FORK,AL		THR,W,OFF		
0x_7	WREG,L	MACX,7	RREG,7	WREG,7	MWRITE	HOP,AB	COND,W	COND,D	RREG,Z	BRL	BSL	ZB3	FORK,AB		THR,W,ON		
0x_8	END	MACX,8	RREG,8	WREG,8	RREG,MA	WREG,MA	CLRA	RREG,FS0	ACOFF	BSET	OR	NOR	FORK,MA		GRIP,OFF		
0x_9		MACX,9	RREG,9	WREG,9	RREG,MX	WREG,MX	SETA	RREG,FS1	ACON	BCLR	XOR	XNOR	TERM		GRIP,ON		
0x_A		MACX,10	RREG,A	WREG,A	RREG,MY	WREG,MY	CLRB	PVFLG	ACLR	BINV	IMP	INH	DUPE		IFC,OFF		
0x_B		MACX,11	RREG,B	WREG,B	RREG,MP	WREG,MP	SETB	GPFLG		BTST	CMP	NAND	TRMD		IFC,ON		
0x_C		MACX,12	RREG,C	WREG,C	RSTMA	REDO	CLRFLT	CLRC		ADD	SUB	AND	RREG,AP		THR,A,OFF		
0x_D	LDW***	MACX,13	RREG,D	WREG,D	JMP***	JSR***	SETFLT	SETC		ADD,W***	SUB,W***	AND,W***	RREG,EP				
0x_E	LDB**	MACX,14	RREG,E	WREG,E	PAGE**		CLR**	FSEL **	**	ADD,B1**	SUB,B1**	AND,B1**	HALT		IDLY**	DROT,R**	
0x_F	WRMAC**	MACX,15	RREG,F	WREG,F	PDEF**		SET**	COND**	LSET**	ADD,B2**	SUB,B2**	AND,B2**	RESET		TDLY**	DROT,L**	

Base CPU Flag IDs:			
Numeric	Mnemonic	FS0 Bit	Description
0	T	b0	Always True
1	F	b1	Fault
2	Z	b2	Zero
3	N	b3	Negative
4	V	b4	Overflow
5	C	b5	Carry
6	M	b6	Memory Address Fault
7	W	b7	Memory Write Fault
8	O	b8	Memory Cell Owner
9	S	b9	Strong Fault
10	R	b10	Random Bit
11	P	b11	Positive (NOR of 'N' and 'V' flags)
12	D	b12	Temp Register Zero
13	-	b13	Unused
14	A	b14	Private A Flag
15	B	b15	Private B Flag
16-31	G0-G15	b16-31	Group 0-15 (shared or private)

Flags
0 = "T" = Always True
1 = "F" = Fault
2 = "Z" = Zero
3 = "N" = Negative
4 = "V" = Overflow
5 = "C" = Carry
6 = "A" = Group A (shared or private)
7 = "B" = Group B (shared or private)
8 = "M" = Memory Address Fault
9 = "W" = Memory Write Protect Fault
10 = "O" = Memory Cell Owner
11 = "S" = Strong Fault
12 = "X" = Group X (shared or private)
13 = "R" = Random Bit
14 = "P" = Positive (NOR of "N" and "V")
15 = "D" = Temp Register Zero
16-31 = "G0"- "G15" = Group 0-15 (shared or private)
32-63 = Reserved for expansion

Robot Flag IDs:			
Numeric	Mnemonic	FS1 Bit	Description
32	NB	b0	North/Front Sensor 1 (bumper)
33	EB	b1	East/Right Sensor 1 (bumper)
34	SB	b2	South/Back Sensor 1 (bumper)
35	WB	b3	West/Left Sensor 1 (bumper)
36	NS	b4	North/Front Sensor 2
37	ES	b5	East/Right Sensor 2
38	SS	b6	South/Back Sensor 2
39	WS	b7	West/Left Sensor 2
40	NT	b8	North/Front controller 1 (thruster)
41	ET	b9	East/Right controller 1 (thruster)
42	ST	b10	South/Back controller 1 (thruster)
43	WT	b11	West/Left controller 1 (thruster)
44	NC	b12	North/Front controller 2
45	EC	b13	East/Right controller 2
46	SC	b14	South/Back controller 2
47	WC	b15	West/Left controller 2
48	GA	b16	Gripper Active
49	IA	b17	Interface Active
50	RX	b18	Radio Comm.
51	AS	b19	Area Sensor
52	-	b20	Unused
53	-	b21	Unused
54	-	b22	Unused
55	-	b23	Unused
56-63	-	b24-31	Unused

LSET: ALU Loop Mode Bits	
Unary Input 1:	b0-1
Binary Input 1:	b2-3
Binary Input 2:	b4-5
Unimplemented:	b6-7
	00=No Change
	01=Unary ALU Out
	10=Binary ALU Out
	11=Clear Loop

Common settings:	
LSET 0x01:	Loop Unary ALU Output to Unary Input
LSET 0x02:	Loop Binary ALU Output to Unary Input
LSET 0x04:	Loop Unary ALU Output to Binary Input 1
LSET 0x08:	Loop Binary ALU Output to Binary Input 1
LSET 0x10:	Loop Unary ALU Output to Binary Input 2
LSET 0x20:	Loop Binary ALU Output to Binary Input 2
LSET 0x03:	Clear Unary ALU Input 1 Loop
LSET 0x0C:	Clear Binary ALU Input 1 Loop
LSET 0x30:	Clear Binary ALU Input 2 Loop
LSET 0x3F:	Clear All ALU Input Loops

General Purpose Registers	
0-9	Private, per-process registers
A-F	Shared between all processes

7654 3210	Instruction Bit Decoding
000- ----	Instruction proc
001- ----	Register Bank
010- ----	Program counter & Memory
011- ----	Flag & Conditional
10-- ----	ALU
1100 ----	Context Switching
1101 ----	Game Subsystem 1
111- ----	Game Subsystem 2

Defining a new page:
Set registers MX,MY with coordinates
Use PAGE opcode to set page number
(‘F’ flag can be used to verify page use)
Use PDEF opcode with page size specifier to Create page.

JMP/JSR: use PAGE first to set page to jump to, or ‘PAGE 0’ for current page.
JMP/JSR/PDEF will clear PAGE register to 0.

Page size specs:	
0	clear page
1	256x256
2	128x128
3	64x64
4	32x32
5	16x16

Conditional Byte (Inst. word Byte 3/7)								
Instruction word: 0 = 1 opcode byte 1 = 3/7 opcode bytes	Bit 7	Bit 6	Bit 5	Bits 0-3				
Flag: 0=auto sel., 1=selected								
Compare against:0=false, 1=true								
Breakpoint set:0=false, 1=true								

Action: (squ grid)	Action: (hex grid)	Action: (tri grid)
0000=NOP	0000=NOP	0000=NOP
0001=ROT,RR	0001=ROT,RR1	0001=ROT,RR
0010=ROT,RL	0010=ROT,RL1	0010=ROT,RL
0011=ROT,RB	0011=ROT,RB	0011=ROT,RB
0100=ROT,AF	0100=ROT,RR2	0100=ROT,PF
0101=ROT,AR	0101=ROT,RL2	0101=ROT,SL
0110=ROT,AL	0110=ROT,AL	0110=ROT,SR
0111=ROT,AB	0111=ROT,AR	0111=ROT,PB
1000=REDO	1000=REDO	1000=REDO
1001=TERM	1001=TERM	1001=TERM
1010=LRET	1010=LRET	1010=LRET
1011=LBRK	1011=LBRK	1011=LBRK
1100=RET	1100=RET	1100=RET
1101=	1101=	1101=
1110=	1110=	1110=
1111=	1111=	1111=

CPU ² Robots Game Subsystem	
THR,N,OFF	Disengage North Thruster
THR,N,ON	Engage North Thruster
THR,E,OFF	Disengage East Thruster
THR,E,ON	Engage East Thruster
THR,S,OFF	Disengage South Thruster
THR,S,ON	Engage South Thruster
THR,W,OFF	Disengage West Thruster
THR,W,ON	Engage West Thruster
GRIP,OFF	Disengage Gripper
GRIP,ON	Engage Gripper
IFC,OFF	Disengage Interface Connection
IFC,ON	Engage Interface Connection
THR,A,OFF	Disengage All Thrusters
IDLY**	Instruction Cycle Delay ## x 1 Cycles
TDLY**	Tick Delay ## x 1 Game Ticks or Frames
RREG,IFC	Read from Interface
WREG,IFC	Write to Interface
RREG,RD	Read from Robot Display Register
WREG,RD	Write to Robot Display Register
DMODE,C	Compass directions, North/South/East/West
DMODE,R	Relative directions, Front/Back/Left/Right
RROT,R**	In Relative mode, rotate right ## degrees
RROT,L**	In Relative mode, rotate left ## degrees

** Two-byte opcode, second byte is data value.
*** Three-byte opcode, second and third bytes are data values.

CPU ² Robots Alternate Direction Mode	
THR,F,OFF	Disengage Forward Thruster
THR,F,ON	Engage Forward Thruster
THR,R,OFF	Disengage Right Thruster
THR,R,ON	Egage Right Thruster
THR,B,OFF	Disengage Back Thruster
THR,B,ON	Engage Back Thruster
THR,L,OFF	Disengage Left Thruster
THR,L,ON	Engage Left Thruster

Game Concept: CPU² Robots (strongly influenced by the game 'Robot Odyssey' published in 1984 by The Learning Company).

This game mode is specifically designed to be a 2-D programming version of a Robot Odyssey like game. Please refer to the original Robot Odyssey game (or one of its clones, DroidQuest or GATE) for basic reference information.

Similarities and differences between this concept (CPU² Robots) and Robot Odyssey (R.O.):

1. R.O. uses 20x12 screens, with five levels (plus intermediate auxiliary areas) with up to 32 screens per level. R.O. uses screen switching (aka page-flipping). My design would have CPU² Robots using a 256x256 area, where rooms can be defined as custom-sized non-scrollable rooms or rooms that scroll horizontally or vertically. All of the rooms would be fit within the 256x256 area. Extra data would determine how rooms connect or how doors are interacted with.
2. R.O. has four robots. This game will also have four robots. The CPU² processors would be fully simulated in software. The first three robots will be silver, but will be color-coded with red, green, and blue corners, and the fourth will be black and purple. That robot will be the last robot the player receives and, as the security-enhanced robot, it has a coating giving it immunity from EMP weapons and EM fields. It also has its external grippers enhanced with EMP weapons.
3. R.O. uses logic I/O signals for bumpers, thrusters, radio I/O, and grabber controls. This new design will use flags for input sensing, opcodes for enabling/disabling outputs, and a custom network module for radio I/O. Each robot can use one or more processes for simultaneous control of different aspects of the robot.
4. No map parts of R.O. will be copied; the entire CPU² Robots map will be created from scratch, with some similar, but updated concepts and all areas entirely new and distinct. The R.O. city was named Robotropolis; For now, the CPU² Robot city will be called Robostadt as a placeholder for a possible different name. The Robostadt city is one of four quadrants of a mega-city. The entirety of this game is played within the Robostadt quadrant. The other three unnamed quadrants could be potential future map/story/puzzle expansions.
5. R.O. uses logic gates (that are recursively combinable into chips) to control the robots. CPU² Robots use 16x16 or 32x32 programming pages and will have opcode access to all of the standard features of the CPU. Some of the programming areas may be pre-programmed with complex movement programs (like wall following). Some puzzles may require a robot to download and write a new program area for solving the puzzle.
6. R.O. robots were limited in functionality to movement, wall sensing, grabbing and releasing objects, and simple binary signal communications between robots. CPU² Robots will have that functionality along with more complex features such as allowing a robot to "plug" into an interface and performing more complex logic solving or manipulation via programming.
7. R.O. robots have a battery which allow a limited amount of thruster operation before needing to be replaced or recharged. CPU² Robots will have a battery level. This battery level could be used similarly for thruster power or could be used more broadly for CPU cycles.
8. CPU² Robots will have a hardware configuration screen separate from the programming area for various I/O modules such as sensors, shields, alternate movement mechanisms, and filters. Some of these slots will have CPU flags and/or opcodes associated with them for inclusion in programming.
9. Because of the structured design of the robot programming interface, rather than have item storage inside each robot, in this game, the player will have a backpack, accessible by pressing the 'B' key. This backpack can be used to store the robots, as well as any sensor modules, batteries, tokens, keys, and other objects that are not needed at the moment. Optionally, when entering an area with no need for robotic operations, the game can be set to auto-drop robots into the backpack so that the player doesn't have to constantly carry the robots around by hand. When the robots are in the backpack, an icon will appear at the bottom right of the screen for each robot. The player can click on the icon to access the robot or double-click on the icon to remove the robot from the backpack. The player can access a robot interface) by coming in contact with the robot and pressing the 'E' key. Once inside the interface, the player can use the 'P' key for the periscope, the 'B' key for the backpack, and the 'C' key for the robot configuration screen.
10. CPU² Robots will have a 'Robot Display' register which can receive up to eight characters. When the register is written to, on the map, the robot will briefly have a speech bubble containing the characters. The character set is a combination of ASCII and a robot-gibberish character set. Robots that are direct in contact with one another can send a display message from one robot to the other. When a message is successfully sent, the receiving robot will repeat the message in a speech bubble. The robot display might have a purposeful lag or cooldown time associated with using it to prevent it from becoming a replacement for radio communications.
11. Power cell chargers are used in R.O. to recharge cells. Some of the rechargers are locked behind simple puzzles. In CPU² Robots, battery cells have a form of monetary value, and cells are expended and disappear once used inside a robot. If the player has a robot run out of power, a variety of mundane tasks and simple puzzles are available for the player to earn battery cells. The player should never face a situation where game progression ends because the robots have had their power depleted. There might also be other non-puzzle tasks for the robots to perform to earn money.
12. The four robots the player programs in CPU² Robots will not be simple machines in this game like they are in R.O. They will have NPC dialogue and be featured in some parts of the overall story. Their primary dialogue purpose (outside of the story arc) will be to explain new environments, orient the player as needed, and remind the player as to what steps need to be taken to move forward with the game progress; crucially, this includes using plausible NPC dialogue to better explain a puzzle in a way that couldn't be done passively. This will allow more interesting and complex puzzles to be implemented in the game in a way that isn't game-breaking or overly confusing for the player. Other aspects of NPC dialogue could be explored; the different robots might have different personalities and offer slightly different takes on how they perceive and report things. Some NPC dialogue could be presented as an optional drill-down dialogue tree that allows the player to stop receiving dialogue about a specific topic when they are satisfied with what they have received so far. Possible idea: Some of the story arc discussions might be presented as cut scenes designed to look like a static comic book page with panels. Sequential panels and word balloons will appear with dynamic text filling in to simulate talking by the characters. A rough example I am thinking about is the original 2001 Max Payne video game, except with word balloons appearing and filling in as the conversation takes place. If a cut scene is not appropriate, word balloons might appear from the individual robots either inside or outside the backpack. One rule would be that a cut scene would never appear if any of the robots are running a program, but individual dialogue might still show up. Verbal dialogue/voice acting will not be used in this game.

13. The robots in R.O. had cardinal direction based (north/south/east/west) directional thrusters, wall bumpers, and other sensor objects. The CPU² Robots can switch between compass (cardinal direction) mode and relative mode, which renames thruster, bumpers, sensors, etc. to front/back/left/right, and adds rotate left and right opcodes for the robot movement. The compass direction mode is locked behind needing a compass module. In the relative thruster mode, the rear thruster is stronger and moves the robot at double normal speed, and the left, right, and front thrusters push at regular speed.

CPU adjustments and other information for this game:

1. CPU will be 32-bit, with a 64-bit macro area.
2. A configurable page mode will be added to implement configurable multi-size program pages. A 'page' register segment will be added to the MAR and PC registers, as well as the mini-stacks for the LOOP and JSR subsystems.
3. The programming will utilize most of the current standard opcodes 0x00 thru 0xBF. Obvious additional opcodes used for robotic functionality can be defined in the 0xE0-0xFF areas. A non-shared map memory (one 256x256 area per CPU) will be used for definable page areas for programming. The user can select which specific program area is default for each robot, allowing storage and reuse of programs. The robot will also have the programmatic ability to switch execution between pages. The current program counter consists of an 8-bit X coordinate, an 8-bit Y coordinate, and a 2-3 bit direction. The program counter for this new mode will be variable, depending on the size of page being accessed, with a total of 16 X/Y/page bits plus 3 direction bits.
5. New flags will be defined for bumpers, gripper sensing, interface, radio, and remote control. Current flag subsystem will have standard CPU flags 0-31 remaining how they are configured, and flags 32-55 configured for CPU² Robots. Flags 56-63 are unused.
6. Each robot can utilize multiple processes. The initial process will start in the lower left corner (relative coordinate 0,0) of the page assigned to the robot. Processes can be forked (duplicated) to become dedicated to thrusters and movement, bumper processing, or radio communications.
7. Unlike the CPU clock setup planned for the regular CPU² Grid Wars game mode (which runs at a dynamic speed to optimize viewability of watching the game combat), the CPU²:Robots game mode will have the CPU run at a high rate dependent on the granularity of movement of the robot, such as 64 full process execution cycles of all processes, for each pixel that the robot moves. This is necessary to prevent the robot from missing or lagging on an event such as a wall collision or radio communications. This speed will need to be optimized for the game.
8. Certain pre-defined programs might be loadable into the robots program area to act in a similar was as pre-defined chips in R.O. This could also be extended to allow a robot to simultaneously run processes on multiple areas defined for specific functions such as complex movement, communications, and complex object manipulation. User pages will be numbered starting at page 0, and game-supplied pages will be identified with letters A-Z. Pages will have a 16-character title, with numeric pages editable by the player and letter-coded pages hardcoded into the game. In-game program cards can unlock specific programs. These cards are available from some puzzles or NPC robots in the game. There will be a special spot in the backpack for storing these cards. When starting to use a new page, space will be automatically allocated from the page memory corresponding to the size of page requested. A form of auto-layout will be used to optimize space in the overall program area map.
9. The thruster on/off opcodes are simply setting or clearing the 'Engage Thruster' flags, which would be what actually controls the thrusters. Each robot individually has a radio communications flag (1-bit register) that it can toggle (to send a high or low signal), but when reading the flag, it effectively is a logical OR of the four robots radio flag registers. A robot will not be able to distinguish whether it is sending a radio high signal or if another robot is, so programs will need to be written to ensure the radio flag is cleared when not sending in order to receive properly. The read-only gripper and interface flags are set to true when the gripper has grabbed something or the interface has been connected, respectively.
10. The interface register could be padded byte-wide or 32-bit, or might possibly require multiple reads to obtain all of the data depending on the puzzle design. Some puzzles might require both transmitting and receiving data from an interface.
11. For some of the more advanced puzzles, newer machines need to be programmed in similar ways as a robot, except they would use a hexagonal or triangle program grid instead of a square grid. The drone might use a hexagonal grid instead of a square one, to indicate a more advanced or exotic technology. Triangle grid robots and machines would only be used for the most difficult puzzles and obstacles, since the programming would be so challenging. The triangle grid is likely limited to 16x8 or 32x16 instead of the usual 16x16/32x32 due to its layout.

Note: All story and lore ideas presented on these pages is just my ideas used to ultimately flesh out puzzle concepts. All aspects of the story and lore could change or be discarded if it doesn't fit with the game direction.

Robot puzzle concepts and ideas:

Maze traversal using different algorithms
Inter-robot radio communication
Environment counting & sequences
Passing objects between robots
Recording and replaying sequences
Robot pushing or pulling objects to allow passage
Downloading programs which give you ability to solve puzzle
Robots accessing interfaces to obtain puzzle solution or hints
Utilizing a small programmable drone for hard-to-access or hazardous areas
Robots controlling a machine to manufacture a needed sensor or device
Noisy environments requiring radio communications with noise-resistant protocol
Hijacking/reprogramming other NPC robots or machines to overcome obstacles
Programming robots for mundane tasks to earn money for advanced programs, information, or sensory objects.

Temporary robot upgrade possibilities:

* Upgrade robot to 1/4-size drone chassis for confined space access. Two robots will be upgraded for this: one to a drone body, another will have its gripper swapped with a drone port. The gripper ON/OFF functions will be used to launch and land drone, and the drones programs start when it is launched and terminate when it lands.
* Upgrade gripper to laser to burn away weak walls, debris, and garbage.
* Add directional EM shield and EM sensor to robot to move robot through dangerous area with EM rays. If directional EM shield is activated, corresponding bumper and thruster will not be functional, so the shield might have to be selectively powered on and off to get through an area. EM sensor will indicate which direction EM rays are coming from, and puzzle might occasionally or frequently change EM ray direction.
* Upgrade replaces thrusters with wheels or tracks for areas where thrusters can't be used due to explosive gasses. Wheels or tracks might change dynamics of moving the robot around.

Puzzle Obstacles:

Filter beam: Programmed to only allow certain entities to pass, such as only robots, only humans, or only sentries. Beam will be made up of individual colored elements which indicate restrictions.
Robot guards, turrets, directed EMP, locked doors, laser walls, electric arcs
Hazardous areas such as toxic, high-EMP, windy, seismic, icy, unstable
Moving objects such as walls, elevators and other robots
Invisible walls or paths requiring robot sensors to navigate
Pressure plates, wall switches, pushable blocks
Movable mirrors to redirect lasers
Access devices: Keys, Tokens, Tickets, and Passes
Robots can be drained of power with EMP, EM rays, or electric arcs. Battery cells can recharge a robot.
Robots can be damaged by EMP, strong EM rays, lasers, electric arcs, and turrets. Damage will cause thruster movement and gripper activation to progressively slow and expansion slots to become non-operational. Places will be available for robot repair, but will cost repair tokens.
Places emanating a certain radiation that will prevent the shrink ray from operating.
Certain places will be hazardous to a human outside of a robot.
Certain places will be hazardous to a human even inside a robot.

Game Concept: CPU² Robots (Continued)

Puzzle rewards (for direct use or trading):
Wafers (flat currency, crafted from waste silicon wafers)
Energy cells and extended energy cells
Bronze, silver, gold, or platinum repair tokens
Robot upgrade authorizations
Zone transit passes
Surface Internet VPN hourly passes
Sensor modules needed for other puzzles or game progression
Programs or security codes needed for other puzzles or game progression

Possible game map design:

The Robostadt is the first of four semi-isolated quadrants of an unnamed megacity, with all of the levels of this game contained within this quadrant. The other three quadrants will be reserved for potential game expansion. The overall map will consist of eight to ten levels for the regular gameplay. These levels will alternate between levels with tutorials, story-related lore, and simple puzzles to learn some aspect of assembly programming and levels that have challenging puzzles that require programming features that the player has learned.

* Robostadt slums – cramped, dirty, filled with lower-class robot workers.
Tutorials: thruster & gripper opcodes, conditional bumper flag functionality, simple loops.
Street design style: ornate brownstone stoops

* Robo industrial zone – Bot Proving Grounds, Droid Ore Refining, Chassis Forge, Base Logic, Lasenrich.
Puzzles: simple mazes with varying wall configurations and grabbing objects

* Midtown – small robohouses filled with middle-class robots.
Tutorials: fork opcodes, radio flag, basic register access, ALU functions, nested loops.
Street design style: mix of small 5-over-1 apartments and row houses

* Commercial zone – Silicon Crucible, ElectroPower plant, BotLogic Data Compiling, RoboExpand.
Puzzles: timing puzzles, added sensors & functions, and inter-robot communications

* Upper Robostadt – large robot condos with lots of security.
Tutorials: Interface & robot display I/O, memory & dupe opcodes, advanced ALU functions.
Street design style: modern condominium buildings

* Robostadt research -- Logic foundry, Robolithography, Nanoscale assembly, Automaton training.
Puzzles: Interface puzzles, drone puzzles, black-box program puzzles, search and sort puzzles.

* Skyline district – Penthouses with lots of exotic robots and machines.
Tutorials: Macros, subroutines, page functions.
Street design style: Interconnected penthouses hanging from the bedrock ceiling.

* Robogov district – Quadrant control, Supercompute simulator, Mainframe Collective, 'Botversity.
Puzzles: hybrid interface/maze puzzles and other advanced puzzles (regular end of game)

* Abandoned roboslums – filled with lost, corrupt, or criminal robots and organized crime rings.
Tutorials: Miscellaneous remaining opcodes.
Street design style: makeshift buildings built into cave-like areas.

* Forgotten ruins – nameless areas, with hazards and increasingly difficult challenges.
Puzzles: Miscellaneous difficult puzzles.

Room configurations:

1. A single fixed room between 3x3 and the maximum static on-screen room size (such as 24x16)
2. Multiple identical-sized rooms with page flipping between rooms.
3. A long horizontally-scrollable room, 3-16 high and width more than the on-screen room width.
4. A long vertically-scrollable room, 3-24 wide and a height more than the on-screen room height.
5. A room larger than 24x16 in both dimensions, with both X any Y scrolling.

Sensor ideas:

Types: Area sensor: Report if an object is in the area (visible on the screen).
Contact sensor: Report if the robot is in contact of the object.
Directional sensor: Report the direction of the object in relation to the robot.
Gripper filter: Restrict the gripper to only picking up this type of object (object sensors only)

Specific object sensor: A sensor detecting a specific object.
"Any object" sensor: A sensor detecting any loose object.
Sentry sensor: A sensor detecting sentry robots.
"Any robot" sensor: A sensor for detecting any robot.
EM ray sensor: A sensor for detecting a strong electromagnetic ray presence or source.
Turbulence sensor: A sensor for detecting wind or other turbulence.
Compass sensor: A sensor for detecting the north pole direction (magnetic declination corrected)
Acceleration sensor: A sensor that will detect movement or movement direction.
Interface sensor: A sensor that will detect an interface location.
Track sensor: A sensor that will detect the presence or directions of a hidden guidance track

Repair token tier & repair time risk metrics:

	Repair token:	Platinum	Gold	Silver	Bronze
Slow repair (20 minutes):		N/A	0%*	5%	10%
Normal repair (10 minutes):		0%*	5%	10%	25%
Fast repair (2 minutes):		5%	10%	20%	50%

* includes 1 free follow-up fast repair for the same robot.
Percentage = chance or speed of degradation leading to possible future failure.

Filter beam/field colors:

Red: Robots not allowed
Blue: Humans not allowed
Purple: Humans only allowed if carried within a robot
Green: Health/capability check: Entities must move across the beam or through the field on their own.
No carried/pushed/pulled entities. Excessively damaged robots or injured humans are also not allowed to cross.
Green/Orange*: No assistance. The player must press the associated start button before the robot crosses the green beam and must keep button pressed until the robot crosses the orange beam, otherwise the orange beam will turn red and block the robot from exiting.
Black: Nothing is allowed to cross the black beam/fields, although the black beam/fields will be disabled if the robot successfully crosses the green/orange beam/field pair.
Any of the beams can be disabled with optional switches. Fields or beams may also indicate a one-way requirement with arrows or triangles.

* Robot description:

The four robots are compact cargo bots (1.25m, roughly cube shaped), used to move materials for the research department. Three of them (silver with red, green, or blue trim) have an integrated shrink beam which will shrink larger low-density cargo to fit within their cargo compartments. Items in a shrink beam will also become an equivalent fraction of their weight. These cargo bots were among the first of the generation 3 robots to be created. The shrink beam will shrink a human or aluminum bipedal robot to about ½ height and a steel bipedal robot to about ¾ height.

* Cargo bot roles:

The red, green, and blue cargo bots (Sal, Viz, and Lev) work as a team to transfer robot-building materials between sites. During normal work days, the three robots spend all their time in the research sector moving materials between areas. Once a week, they have a special time-limited task to transfer materials between the industrial and research sectors. During these operations, they work as transporters: One transfers raw materials from Refining & Reclamation to Robolithography for chip making. Another one transfers new CPU modules and other chips from Nanoscale Assembly to Base Logic for low-level ROM programming and chassis setup. The third transfers dismantled modular robot chassis cores from the 'Bot Proving Grounds to Automaton Training for high-level training and personality setup.

For the special operations, a fourth robot (Dux) is also brought in to monitor and provide limited security for the three robots and to transport high-security or prototype materials, if required. That robot (black with purple trim) has an EMP-hardened coating and EMP weapons installed on its grippers, but is otherwise an identical model as the three. Because the EMP weapon takes up the slot usually used for the shrink beam, the black robot has a modular carrier strapped into its cargo compartment that has a built-in shrink beam for picking up small amounts or low density materials, and a detachable remote console. When it is not on one of these special transport missions, the black robot is providing security for some of the research areas.

* the black robot's upgrade:

The black robot has a full-chassis EMP-hardened coating and EMP weapons installed. The coating is very rare due to it not being available in quadrant 1, and can only be obtained from the third quadrant. Due to strict quadrant isolation requirements, the black robot's chassis had to be transferred, without its CPU module, through a heavily-restricted transfer point down into Q3 for several weeks to have the ferro-ceramic coating applied. After the chassis was returned, the CPU module was reinstalled and the black robot then had to undergo a short quarantine and Bot Proving Grounds re-testing. One of the Q1 research robots divulged to the black robot that the payment to the Q3 government for this upgrade was 10 prototype generation 4 androids. A very expensive upgrade, indeed.

Critical story elements to address:

1. How did the human get into the robot city? And why?
2. For the four robots that the human interacts with, how did they get into a state where they need assistance?
3. How does the human help resolve the four robots problem, and what was the actual problem caused by?

Other minor story elements:

1. Humans in the robot city: how they survive and why there are allowed to be there.
2. Feasibility of the human being able to program and modify the robots.
3. Explanation for why the human can carry multiple robots and also be able to ride inside of the robots.

Current limitation: the detailed story arcs and the puzzles need to be developed at the same time as game creation.

Possibly intro story concept:

After being given the archeological research papers from their deceased uncle's estate, Ash tried to figure out for themselves what their uncle was always so unwilling to describe in anything more than extremely vague and unsatisfying language. Digging through the nonsensical notes with strange boxy figures scribbled in pencil in the margins, Ash ultimately found a map leading to the location of a cave in one of the documents. Having recently quit their job, Ash decided to drive out to find the cave and spend a week or so exploring and camping.

Ash followed the dusty, faded map as best as they could, with roads becoming rougher and more impassable as time went on, some almost appearing to have been purposely altered to be as inhospitable as possible. As sunlight was beginning to fade, Ash finally arrived at where the cave should be, only to find nothing but razor-sharp blackberry bushes covering the side of a hill. Using long leather gloves and a machete to chop a path through the bushes, Ash eventually came across a heavy teak panel partially covered by dirt and firmly embedded in the 30-degree slope of the hill. Using their gloves to dig the dirt away, Ash managed to pry up the upper edge of the hinged panel and expose a small cave opening, with a rope attached to the panel and a rope ladder leading diagonally down into the deep, black darkness. After climbing down into the hole, Ash looked around with a flashlight and found what looked like an old mining lift with a sign next to it. The sign had an arrow pointing up toward the ceiling and stated, "Extremely Important: Use the rope to keep the door closed." Without really thinking it through, Ash used their full body weight to pull down on the rope dangling from the ceiling next to the rope ladder, and the outside wooden panel over the cave entrance swung closed with a heavy thud. Realizing they were now likely stuck in the cave, Ash carefully examined the lift and decided to explore further. They stepped into the lift and pulled the lift door closed. After a bit of a delay, Ash tugged on a large metal lever to start the lift descending. Ash was taking the lift down to what they thought was their uncle's secret archeological cave site when the lift suddenly started repeatedly shuddering and falling, screeching and slowing down, then shuddering and falling again. Ash was knocked out when the lift finally hit the bottom of the shaft.

As Ash slowly regained consciousness, and while still in a daze, they watched a large figure cloaked in black fighting a gray figure on what looked like a dimly lit street corner. The gray figure dropped a box as they were being dragged away. Once finally able to see clearly and stand up, Ash marveled at how the area was made to look like an inner-city neighborhood, except with a high rock ceiling – it almost looked like a chunk of city buildings had been teleported into the cave. After a muffled sound caught their attention, they walked over to examine the large silver box dropped by the gray figure. The box appeared to be a slightly trapezoidal-shaped aluminum box and had what looked like backpack straps on the largest surface and handles on the narrow sloped sides. Suddenly, a tiny mechanical voice told them to take the "carrier" and quickly find a place to hide. Seeing what appeared to be another black figure in the distance coming towards them, Ash reluctantly grabbed the metal box and ran around the underground neighborhood trying to make sense of it all: standing on both sides of a wide cobblestone path, a long section of various stooped brownstone apartments seemed to go on and on into the distance, lit by street lamps, apartment windows, and dark blue ambient light shining up onto the light-gray rock ceiling, possibly coming from what Ash suspected were hidden lamps mounted along the tops of the buildings. The lack of a wide central street gave the area a really cramped feel, and the lack of any cars made it feel abandoned. Once Ash found a place to hide in a side alley, the voice told them to open the box and remove them. Ash cautiously reached in and removed a large cube-shaped object but suddenly dropped it on the ground in surprise as it quickly grew into a large silver and green box, while rising and hovering about a half-meter off the ground. After a few seconds, a cylindrical head lifted up out of the top of the cube, and it introduced itself as "Viz." Before Ash even said a word, Viz, sensing that this may be this human's first visit, proceeded to give them a brief history of the robot city and its inhabitants. Viz will be the first robot that Ash will befriend. Ash's first task will be to help Viz locate their other three robot friends as they ascend through the robot city, Robostadt.

* Cargo bot incident:

The four cargo bots were tasked with a cross-zone delivery of a double order. The mission directed the robots to load up and deliver two loads instead of just one. The red bot was assigned to raw materials, the blue bot was assigned to robot chassis, the green bot was assigned to CPU module delivery, and the black robot was delivering gen-4.1 mega-CPU modules. The first delivery was uneventful, with all bots delivering their loads successfully.

While the robots were going back for their second loads, the green robot suddenly found itself surrounded by unknown robots in the midtown district, and being pushed into a back alleyway. It sent a radio alert to the black robot, which was at the other end of the same district. The black robot, after directing the other two robots to hide, spent time searching the length of the midtown district unsuccessfully for the green robot. The black robot then used a comm terminal to contact the research department to report the incident. While waiting for a response back, the black robot, as well as the other three, suddenly had their CPU-chassis interfaces remotely deactivated, rendering them unable to move or use their other external peripherals.

A criminal bot and its thugs grabbed the green robot and attempted to also grab the black bot, but a homing beacon was activated when they opened the black robots cargo compartment. It grabbed the carrier and left the black robot locked in a back room. The blue and red robots had successfully hid – the blue robot was in the commercial district and the red robot was in the Industrial district. To make it easier to carry, the criminal bot reconfigured the carrier into its backpack configuration using integrated straps on the carrier.

Because the cargo bots missed their mandatory 3-hour check-in after the special transportation task, their special inter-zone security access has been disabled and they are now stuck on whatever level they were on. Ultimately, after an incident that disabled the criminal bot, the player obtains the backpack and the green robot. The bots, with specifically designed shrinkable chassis, will shrink enough (down to 25cm cubes) to allow all four of them to fit within the backpack, attached with magnetic pads. While there, they can continue to communicate with the player using their vocal circuits and be programmed with a wireless console interface.

The CPU-chassis interfaces are still disabled on the four robots, so they cannot move on their own – they will have to rely on the player to either carry them in the backpack or program them for movement using their grid interface.

Possible cargo bot story timeline: (1-3 are before Ash)

1. Black cargo bot (Dux) gets chassis upgraded with anti-EMP coating, gets recertified in the Bot Proving Grounds.
2. In the course of routine robot business, it's Dux's turn to pay the four cargo bot's shared condominium tax to the government. Dux submits the payment.
3. The incident with the criminals occur and the cargo bots lose their movement functionality.
4. Ash's intro story occurs. As Ash arrives on the old lift, they witness a black sentry arresting another figure. That other figure is the 'criminal' bot that targeted the cargo bots. Ash, rescues the green robot, Viz, in the slums.
5. Ash meets the bot proving grounds inspector and is told that only unofficial tests can occur because the official trainer is missing. Ash completes the course and rescues the red bot, Sal, in the Industrial district.
6. They reach a government field office at the far end of midtown, and a government bot looks the information up and tells them that the cargo bots were disabled due to non-payment of taxes, and that they would have to talk to their bank to find out why their payment didn't go through.
7. Ash continues on through the commercial district and rescues the blue robot, Lev.
8. After working through the commercial district, Ash reaches the Robobank branch. The bank teller indicated that Dux's payment never went through because Dux is currently decertified and needs to get re-certified by the government to unfreeze their bank account. That can only occur with a special authorization from the central mainframe in the government district.
9. In Upper Robostadt, Ash locates the 'criminal' bot that cornered Vix and discovers they are a private tax collector that was recently released from jail. The tax collector tells the cargo bots that it was going to collect the missing condominium tax when it panicked and fled when they saw Dux's homing signal and thought it was calling a bunch of black sentries, since Dux and the sentries have the same black coating and weapons and the tax collector suspected it was under surveillance by the government. The tax collector bot gives Ash a key and directions to unlock the room that Dux is locked in down in midtown. Ash goes and rescues Dux.
10. Ash continues up through Upper Robostadt and the Research sector.
11. A research bot gives the cargo bots information that the recertification occurs at the Bot Proving Grounds, but that the official training bot was in an accident a week prior and was deactivated. Even with the authorization, they would have to wait for a new training bot to be approved.
12. Ash continues up through the Skyline district and into the Robogov district.
13. Ash and the cargo bots finally reach the central mainframe in the Mainframe Collective, and discuss Dux's decertification. The central mainframe realizes a mistake has been made somewhere, and issues Dux a recertification authorization, but also mentions the incident about the official training bot. The mainframe examines the four cargo bots grid programming logs and is impressed by the improvised programming that the human has created. The central mainframe grants Ash a temporary official trainer license for the Bot Proving Grounds.
14. Ash and the cargo bots go back down to the Industrial district and show the Inspector bot Ash's trainer license. The inspector lays out the recertification rules: to go through the whole course without a single fault or reset, the restrictive time limit for completing the course, and that even a single failure will mean having the restart the whole course over again. Ash is allowed as many test runs as necessary to program Dux for the recertification.
15. After Dux successfully completes the course, the inspector bot submits the recertification to the government, and tells Dux approval will take a few weeks. Because of the line of bots waiting to be recertified, the inspector bot urges Ash to (optional) recertify the other bots waiting in line so that they don't have to wait weeks in line for the new permanent trainer bot. It indicates that the other bots employers might be willing to offer rewards for quick recertification for the bots.
16. Ash completes the extended non-cargo bot story arc and puzzles.
17. When Ash has finished all the missions and wants to leave, they go up to the secondary lift in the government district. The lift normally has a big robot to supply the big surge in power needed to raise the lift. Instead, the four cargo bots have to plug themselves in to provide that power surge. Everyone says goodbye as Ash steps into the lift and the lift rises up until Ash is gone.
18. Just after Ash leaves, Dux receives notification that their recertification has been accepted, and then receives a notice that their tax payment has been approved. Suddenly the bots are able to move again. Dux admits that they have been able to move on their own since right after the Bot Proving Grounds but that they felt sorry for their other three friends and decided to just play along.
19. The bots move off-screen, then Dux reappears and goes and grabs the dropped backpack, then disappears again.

*** Humans in Robostadt:**
Humans are uncommon in Robostadt, but they do occasionally pop up. Most robots consider them a curiosity more than a threat, and security robots are programmed to specifically try to monitor and maintain a visible presence around humans during their visits to reduce illegal chip tech. sales between robots and humans. Most humans that know about Robostadt are archaeologists or other scientists that are willing to keep it secret. Some robots practice gardening as a hobby and are delighted to provide food to visiting humans. Toilets and wash rooms are also available at the ends of each residential level for human use.

*** Robot programmability:**
All modern Robostadt robots have their core robot programming contained within their CPU modules. Most robots have a secondary programmable area that is a holdover from when they were undergoing initial unit testing. Some employers utilize the programmable area to provide specialized programs used to aid a robot with repetitive tasks or in hazardous or sensitive environments. Some of the newer robots feature a hexagonal programming grid rather than the usual square one, and some exotic machines feature a triangular programming grid and are the most challenging to use.

*** Robot configurability:**
Your robot friends have a configuration panel ('c' key) underneath their programming screen. This configuration panel shows a representation of the robot with the CPU module locked in the center and a slot each for the hovering, bumpers, thrusters, gripper, antenna, robot display, interface, and several auxiliary slots for sensors and other accessories. For these specific robots, they have an extra slot used by the shrink beam module. That module is welded into the slot and non-removable, but it can be switched on and off. To pull the CPU module, the robot has to voluntarily unlock the slot before it can be removed.

*** Breaking rules:**
Robot jail: robots caught breaking robocode gets taken to robot jail. To recover robot, player will have to pay a fine or break the robot out. Robot guards will be more lenient about humans breaking some rules, but will move the player out of an area or expel them to a lower level. Humans found to be distributing illegal chips will be escorted to the exit to the surface and will be told to leave.

*** Robot identification:**
Most Robostadt robots do not have a machine equivalent to human vision – they can take color snapshots, but they cannot process continuous video in real time. Instead, they have various methods including spacial sensory input that detects thermal, electromagnetic, and piezoelectric or laser reflections, size/density estimation, and other attributes to uniquely identify other robots, as well as identify interfaces, doors, walls, and humans. Each CPU module produced by Robolithography is customized to have a set of harmonics that are always emitted while the CPU is powered on. These harmonics are unique for each CPU module and can conclusively identify the robot. Robot sentries (or other robots with heavy anti-EMP coatings) do not naturally emit these harmonics and while they are identifiable as sentries (because of their non-electromagnetic properties), the sentries cannot be differentiated from one another. Your black cargo robot has this same strange attribute, but it can choose to use its radio antenna to broadcast its harmonics when necessary.

*** Robot starting points:**
You start by obtaining the backpack with the green robot in the Robostadt slums. The red robot is hidden in the Industrial district. The black robot can be seen but is not yet accessible in the midtown district. The blue robot is hiding in the commercial district. Once you get to Upper Robostadt, you get an access key which allows you to free the black robot in midtown. You should have all four robots when you enter the Robostadt Research district.

*** Robot power:**
The robots of Robostadt usually recharge during nighttime hours when factories are not in operation and plenty of power is available. Gen 2 and newer robots are able to utilize energy cells to partially bypass their regular power source and use the energy cell for their peripherals. They are able to adjust the energy cell voltage output to as low as 50% (for slower, more gentle and precise handling capabilities) to as high as 150% (for faster movement and stronger grippers). Most robots have an energy cell slot behind their vocal panels. After an energy cell is depleted, robots can drop them into recycling bins distributed around the city. Some gen 1 robots have a crude and unapproved upgrade installed allowing them to use energy cells. Generation 4 robots (and other robots with a special upgrade) can adjust their peripheral power strength without the need for energy cells.

*** Robot generations:**
The first generation of robots are now considered obsolete, although most still have simple jobs to perform, such as ore processing, material recycling, and simple fabrication. These robots have been deemed too nonstandard for any potential upgrade paths and have been whittled down to smaller numbers due to the dangers of their work environments. The gen 1 robots primarily occupy the slums, with a few residing in midtown or work as housekeepers or maintenance for gen 3 or 4 robots. Regular sentry robots are gen 1 robots, although with extremely rugged chassis upgrades and weaponry. Sentry robots do not have personality programming and are the most primitive functioning robots in Robostadt, although they are AI-trained in robolaw and robot law enforcement. Even the (generation 2) sector police are reluctant to cross paths with the sentry robots. All of the original generation 1 sentries are still in operation. Generation 1 population in Robostadt: 2000 robots

The second generation of robots work in various middle-class jobs such as robot repair, cosmetic upgrades, energy sales, simulation theaters, city maintenance, sector police, e-book stores, and transportation operators. Some upgrades have been developed for gen 2 robots, allowing them to be in jobs related to hazardous environments or requiring increased gripper agility, strength, or faster movement. Gen 2 robots mainly occupy midtown, although a few live in Upper Robostadt or the slums. "Droid" is slang for gen-1 and gen-2 bipedal robots. Generation 2 population in Robostadt: 10,000 robots

The third generation of robots have been developed for more advanced roles requiring increased physical capabilities such as speed, strength, and agility, as well as higher levels of reasoning and creativity. Their faster CPUs give them more cognitive abilities and utilize more advanced AI engines. The Gen 3 robots work in advanced roles such as research, high technology manufacturing, advanced robot repair, city planning, and managing large-scale projects. Core upgrades are uncommon for gen 3 robots due to how new they are, although accessory or peripheral upgrades are readily available. The gen 3 robot neighborhoods are spread between Midtown and Upper Robostadt. The Robostadt government is currently gen 3 bots. Generation 3 population in Robostadt: 20,000+ robots.

The forth generation of robots is the newest one, with very few robots currently in full operation. These robots feature the most advanced AGI engines with the highest-performance mega-CPU modules and nearly unlimited memory. Their official jobs are in consulting for research, commercial, and government sectors. Unofficially, they excel in creating fiction e-books, puzzles as well as simulation performances and video games. Most work in entertainment directed towards the gen 2 and 3 robots. Gen 4 robots live exclusively in the Skyline district. They feature a triangular programming grid interface that is purposely very difficult to master. Generation 4 population in Robostadt: 50 robots.

*** City day/night cycle.**
At some point, humans helped the robots set up a surface camera on an old mining ventilation shaft to obtain a fish-eye skylight picture. Parts of the city use that information to control ambient lighting lamps to emulate the skies color and brightness, with some areas even being able to simulate sunrise and sunset colors and dark stormy days. Most robot workers follow a regular day work shift, evening time off, and nighttime downtime to recharge and run optimization and exercise routines on their robotics and further their AI subsystem training. On the surface, a small cabin was built on top of the ventilation shaft and the camera is disguised on the roof. The shaft still supplies a little outside ventilation for the city.

*** Misc dimensions:**
Cargo robot size: 1.25m cube plus 40x68cm cylindrical head (excl. hat rim width) and 1.5m shoulder to shoulder width. When cargo robots are stowed in backpack or shut down, they retract their heads into their cube chassis area for added protection and space reduction.
Cargo robot compartment: 1m cube interior with detachable ceiling-mounted touch screen.
Cargo compartment door: 1m wide by 75cm tall with simple exterior cosmetic display.
Carrier/backpack size: interior: 27cm x 54cm x 60cm, exterior: 30cm x 60cm x 75cm with a lid on one of the 30x60cm ends. The carrier is powered by a small internal nuclear source and doesn't need recharging.

*** Robot/CPU classes:**
Class 1: Bipedal, <3m height, <1m circular footprint, personality capable: yes, loco: mechanical legs
Class 2: Cargo, fits within 2x2x2m area, pc: yes, locomotion: plasma thrusters and hover pads
Class 3: Fixed/Track, no size restrictions, pc: limited or no, loco: track, wheel, chain, or cable drive
Class 4: Fixed/Non-movable, no size restrictions, pc: limited or no, loco: none
Class 5: Modular, restrictions: use other class size restrictions based on modular chassis in use.

*** Slang:**
"Android" is slang for any bipedal robots.
"Droid" is slang usually for generation 1 and generation 2 bipedal robots.
"Sub" is slang for lower generation robots.
"Super" is slang for any higher generation robot, but usually means generation 4 robots.
"Max" is slang for a security guard or sector police officer.
"Spider" is slang for a sentry.
"Black Widow" is slang for a black sentry.
"Tracer/Tracker" is slang for robots that are fixed to tracks or follow hidden guides.
"Anchor/Anvil" is slang for any robots which plug into computers or machines for use as a CPU source.

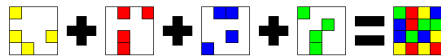
*** Quadrant origination:**
Quadrant 1 was originally a low-grade coal seam (sitting at about a 1.5km depth) that was mined out and abandoned. After surveys were quietly carried out, a copper mine was discovered and a tunnel was constructed from quadrant 1 to what would become quadrant 2, at about 5km away and a depth of about 1km. In roughly the middle of that tunnel was discovered a new coal seam, and a second tunnel was dug out to follow the coal seam to the edge of a large coal mass which would become quadrant 3 at about 500m deep below the top of a mountain. Lastly, as test cores were being drilled in quadrant 1 to determine ground stability, a large gold bearing quartz formation was discovered deep below quadrant 1 at about 2.5km. A small shaft was dug down into the formation and enlarged at the bottom to become quadrant 4. A large angled tunnel was dug from quadrant 4 over to the intersection of quadrant 2 and 3, and that intersection became a high-security transfer point for materials passing between the quadrants. The original shaft between quadrant 1 and 4 is still in use as a ventilation shaft near the original mining shaft connecting quadrant 1 to the surface.

*** Quadrant manufacturing capabilities:**
Q1: Metal ore processing: silver, lead, nickel, zinc, and copper metals and alloys, uranium enrichment.
Q2: Coal ore deposit: coal, coal coke, pure carbon, and rare earth ores.
Q3: Large quartz/gold deposit: gold ore refining, raw silicon and quartz products. Aluminum* recycling.
Q4: Copper ore deposit: Ores: gold, iron, copper, silver, zinc, uranium, other trace elements.
* Robostadt does not have a native bauxite mine, so excess gold is covertly traded with surface aluminum recyclers and refineries with the assistance of human allies. Quadrant 3 has a hidden surface tunnel disguised as an old mining adit for transferring gold and aluminum with trucks and for human access.

*** Quadrant advanced commercial & research capabilities:**
Q1: Silicon boules & chip production, advanced robot programming, robot chassis, nuclear power.
Q2: Plastics, gasses, carbon composites, rare earth alloys, chemicals, emergency coal power.
Q3: Fiber optics, industrial diamonds, glass lenses, sapphire glass products and ceramic coatings.
Q4: Iron ore refining, steel production, purified water, metal plating, geothermal power.

*** Typical robot hobbies:** E-book reading, solving puzzles, simulation theater, gardening, model building, chassis customizing, social connectivity,

Example advanced puzzle:



You have to get a specific robot program which allows it to traverse a restricted area. The program data is split randomly into four separate pages available for download from four interfaces in the room. Each chunk of data is a 16x16 program page with only a random ¼ of the data spread across the page. Combining the four partial program pages together properly will give you the complete program. The partial pages might be rotated or flipped horizontally or vertically in some random combination that changes (along with the contents of the pages) each time you try this puzzle. Once enough partial pages are downloaded, you need to use a program (or subroutine) that can determine how the pages need to be adjusted to be properly combined to build the complete program. The robot can then run the completed program at the appropriate spot to get through the restricted area. To make this puzzle more challenging, you have a limited time to complete the whole puzzle and run the resulting program. This time limit requires that you simultaneously combine the downloaded segments while you are still moving the robot around obtaining the other segments, which means the program needs to autonomously cycle through the various flip/rotation permutations to find the correct orientation. The final combined program also has to be rotated correctly for the proper 0,0 starting point to be used.

Hints:

- * Each of the four partial pages will have one corner filled and the other three corners empty. This may provide a shortcut that can quickly trim away ¼ of the checks.
- * One page always contains the starting program corner cell as one of its corners. This page may be rotated, but will never be horizontally or vertically flipped. The other three will always be flipped either horizontally or vertically and might also be rotated. This should give you the ability to know how to orient the overall combined program to start properly and which corner is the program start.
- * You can tell if pages are properly aligned with each other if there is no common data point in the 16x16 grid on the pages where non-zero data conflicts between the pages. Once the pages are combined, there should be no empty cells.

Example sort puzzle:

Backstory:

At some point in the game, you realize that all of the first generation robots of Robostadt have had a certain portion of their memories mass-erased, but you ultimately find someone that was undergoing major CPU module repair when the erasure was taking place. After they were again operational, they still had the 'unauthorized' memories. So that they wouldn't be found out by the mainframe as still having those memories and punished, they unlinked them from their own memory structure. They are aware that the memories are still there, but they can't access them. Their CPU recovery also burned out some high bits in their ALU so they are not able to sort the 32-bit memory inDuxes themselves if they wanted to recover the memories.

Puzzle:

They are willing to send (via interface) the random 256 unlinked inDuxes to one of your 'bots, and you have to write a sort utility to sort the 32-bit numbers in ascending numeric order and then send the data back out the interface to them.

Reward:

Once they receive the numbers in the proper order, their low-level memory manager can properly scan for and relink the memories back into their memory database. They can then dump the memories to a high-capacity storage disk you obtained earlier.

Hints:

- * Since the robots do not have enough register space, the numbers will need to be written directly to program pages as raw data.
- * Since there is no speed requirement for reading the numbers from the interface before sorting, that can possibly be incorporated in the sort routine itself, so long as it doesn't slow down the overall job.
- * Any suitable numeric sort routine can be used to sort the numbers, so long as its capable of being programmed with the available opcodes. Some high-recursion sort routines may not run properly.

Example advanced networking puzzle:

Backstory:

You are trying to look up a security door code from a secure storage facility. You will use the data token you received from the previous puzzle. The first part of the data token (bits 0-2) is a 3-bit network address which determines the storage vault you need to talk to. The second part (bits 3-15) is the 13-bit lookup key, which determines if you are authorized to get the information. If the drone ends up going to the wrong storage vault, the data token will be invalidated and you will have to repeat the previous puzzle over again to get a new data token. If something goes wrong, the player can press the alarm button on the wall and the sentries will remove any robots in the network nodes. Every time the alarm button is pressed a 5th time, the data token will be invalidated and the player will have to repeat the previous puzzle. The player and the regular robots are too large to enter the network, so one robot will need to be upgraded to a mini-drone to complete this puzzle.

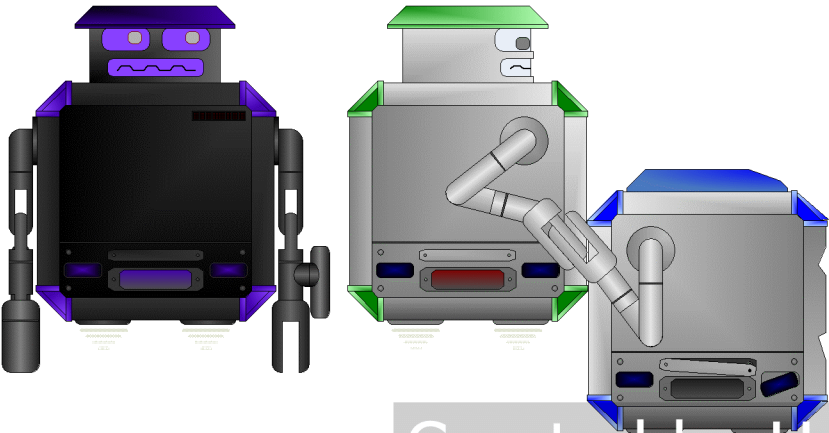
Puzzle:

Access to the storage vault is via an omega network that utilizes an XOR tag address. To traverse the omega network:

1. Send the drone through the entrance. They will be teleported into one of 8 small hallways, each with an interface.
2. Connect to the interface and read the source address from the interface.
3. XOR the source address with the 3-bit destination address to get the XOR tag.
4. Move the robot east. Once it hits the east exit, it will be teleported into the first network node.
5. In the first node, the robot will either be at the upper entrance or the lower one. If bit 2 of the XOR tag is a zero, move the robot straight east to the exit. If bit 2 is a one, move east and north or south to the other exit (diagonal from the entrance) and go through that exit.
6. Repeat step 5 for the second node, but use bit 1.
7. Repeat step 5 for the third node, but use bit 0.
8. Move east to the interface and connect. At the interface, send the data token and retrieve the security door code. If one of the previous steps were not followed correctly, then the alarm will sound and the sentries will remove the drone and the previous puzzle (and this one) will need to be repeated.
9. Once the security code is obtained, reverse the steps (and direction) to move back to the original entryway.

Hints:

- * As with other standard omega network designs, the player can also use destination tag addressing where you only use the destination address (without the XOR), and a zero bit = always uses the top exit, and a one bit = always use the bottom exit. To exit the network, the source address is used.
- * With the drone in a narrow passage, both the north and south (or east and west) wall sensors will be in contact, and the drone will be forced to the center. In these type of passages, a wall sensor signal going low might be a more important detection event than a wall sensor signal going high.



Example challenging environmental puzzle:

Location: Commercial District, Electropower Plant, Central power chamber

Description:

Extreme electromagnetic environment will prevent non-shielded robots, thrusters, hover pads, and shrink rays from operating. Even the black robots cargo compartment isn't quite sealed enough for this area. Without operational hover pads, the robots will be too heavy for the human to move. All four robots and the player will need to cross the area as quickly as possible.

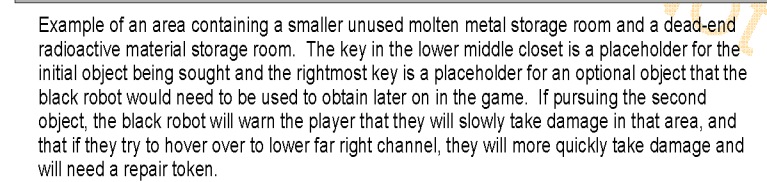
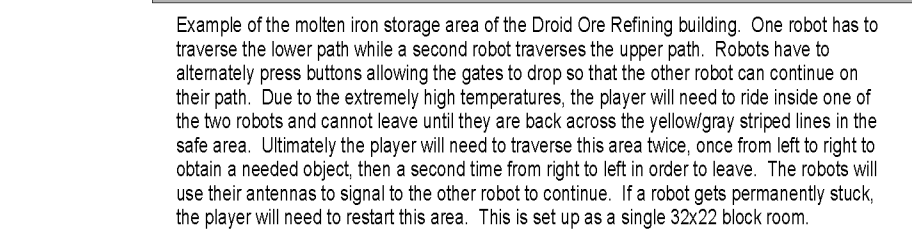
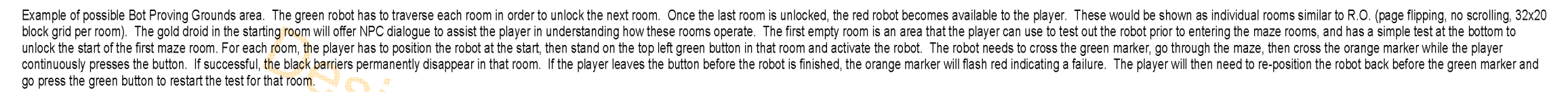
Puzzle:

The black robot will need to be upgraded to use a tank tread attachment* instead of thrusters, and then will need to be programmed to grab and pull each robot individually through a narrow path across the hazardous environment. The silver robots will need to be placed on rolling carts, then shut down and powered off during the process. The human can go across the area, but must be very quick or they will get injured. The backpack will also auto-power off in the area.

Risks:

The black robot will receive damage based on their exposure time in the environment; they may need a repair token after this puzzle. If the stray too far off of the narrow path crossing the chamber, they will trigger a power shutdown which will alert the sentries. If the human gets injured enough by microwave radiation, they may lose the ability to carry robots or the backpack until they get healed up via some rest and relaxation doing tutorials, participating in a robotour, or completing some mundane jobs in a residential district.

* Tank tread attachments and other upgrades can be ordered through upgrade shops in the residential areas, although not every upgrade is available on every level. The upgrade shops have a back room where upgrades can be tested.



Game Concept: CPU² Robots (Continued) – Wall Crawler Program

[illegible]

This is my first attempt at a CPU² program. This should theoretically be a right-wall crawler program, but I currently have no way to test it. The first process creates five other processes: four processes to handle regular wall movement, and one to handle outside corner walls. Register 9 is converted to a group register and is used to keep track of the last activated thruster. Some of the delay values might have to be adjusted or moved around in the code, and this program likely will not be able to handle very erratic wall conditions without further refinement.

This will no longer function because of changes in the opcode table.

[illegible]

This might be a better design for a right-wall crawler program. The cyan process monitors the bumper statuses in the flag register, then uses the shared 'A' flag to signal the other processes as to whether they should handle regular wall movement or outside corner mitigation. Each of the other four processes examine that shared flag to determine which mode they should be in. They then either perform their regular wall movement (and update the shared register '9' if they engage their thruster), or in the alternate mode, they examine the shared register to see if they are the last thruster to fire and if so, they turn on a pair of thrusters to correct for the outside corner. Once any of the wall bumpers make contact again, then the cyan process switches back to the regular mode using the shared flag. The delay opcodes have been left out of this and can be added back in if necessary. This doesn't use any 'redo' conditionals, so it shouldn't be prone to any potential process lock-ups.

This will no longer function because of changes in the opcode table.

	TERM	FORK,RR	NOP	LDB 0x01	RREG, L	VIREG, 0	TERM	THR,W, OFF	CSF,NB, RR	THR,W, ON	RREG, 0		
			NOP	LRET	THR,W, ON	VIREG, B2	THRA, OFF	LRET	NOP	LRET	VIREG, F		
			NOP	THR,E, ON	CST,X, RR	THR,S, ON	LOOP	CST,B, RL	CST,A, RL	CST,X, RL	CSF,NB, RR	THR,E, OFF	LRET
			NOP		LRET	CSF,D, RL	XOR	VIREG, B1	RREG, F	THR,E, ON	RREG, 0	VIREG, F	
		FORK,RR	NOP	LDB 0x01	RREG, L	VIREG, 0	TERM	THR,N, OFF	CSF,EB, RR	THR,N, ON	RREG, 0		
			NOP	LRET	THR,N, ON	VIREG, B2	THRA, OFF	LRET	NOP	LRET	VIREG, F		
CLRA	LRET	NOP	THR,S, ON	CST,X, RR	THR,W, ON	LOOP	CST,B, RL	CST,A, RL	CST,X, RL	CSF,EB, RR	THR,S, OFF	LRET	
CST,D, RL	SETA	NOP		LRET	CSF,D, RL	XOR	VIREG, B1	RREG, F		THR,S, ON	RREG, 0	VIREG, F	
AND	VIREG, B1	FORK,RR	NOP	LDB 0x01	RREG, L	VIREG, 0	TERM	THR,E, OFF	CSF,SB, RR	THR,E, ON	RREG, 0		
TERM	RREG, FS1	NOP	LRET	THR,E, ON	VIREG, B2	THRA, OFF	LRET	NOP	LRET	VIREG, F			
THRA, OFF	CSF,B, RL	NOP	THR,W, ON	CST,X, RR	THR,N, ON	LOOP	CST,B, RL	CST,A, RL	CST,X, RL	CSF,SB, RR	THR,W, OFF	LRET	
	LOOP	NOP		LRET	CSF,D, RL	XOR	VIREG, B1	RREG, F		THR,W, ON	RREG, 0	VIREG, F	
VIREG, F	VIREG, B2	FORK,RR	NOP	LDB 0x01	RREG, L	VIREG, 0	TERM	THR,S, OFF	CSF,WB, RR	THR,S, ON	RREG, 0		
RREG, L	LDB 0x0F	FORK,RL	LRET	THR,S, ON	VIREG, B2	THRA, OFF	LRET	NOP	LRET	VIREG, F			
TERM	PRET	CLR	THR,N, ON	CST,X, RR	THR,E, ON	LOOP	CST,B, RL	CST,A, RL	CST,X, RL	CSF,WB, RR	THR,N, OFF	LRET	
START	FORK,RL	ACOFF		LRET	CSF,D, RL	XOR	VIREG, B1	RREG, F		THR,N, ON	RREG, 0	VIREG, F	

This is an expansion of the middle wall crawler design. This can switch between right and left wall crawling (using the 'X' flag), and terminating the wall crawler processes (using the 'B' flag). The 'A' flag is still used for the movement mode, and shared register 'F' is now used for last active thruster ID storage. This also uses the updated conditional flag functionality, substantially shrinking its size.

Left Wall Crawler:
West bumper activates South thruster #1
North bumper activates West thruster #2
East bumper activates North thruster #3
South bumper activates East thruster #4
North Thruster #1 correction uses South & West thrusters
West Thruster #2 correction uses East & South thrusters
South Thruster #3 correction uses North & East thrusters
East Thruster #4 correction uses West & North thrusters

Right Wall Crawler:
West bumper activates North thruster #1
South bumper activates West thruster #2
East bumper activates South thruster #3
North bumper activates East thruster #4
North Thruster #1 correction uses South & East thrusters
West Thruster #2 correction uses East & North thrusters
South Thruster #3 correction uses North & West thrusters
East Thruster #4 correction uses West & South thrusters

CST, **	Flag is Zero/False
Flag is One/True	CSF, **
▷ Regular program execution ▶ True conditional direction change	
RL	RR

[illegible]

Inst. Word:0x0000F0E

Inst. Word: 0x7200007D
Real Opcode: COND,B
Conditional: CST.ROT.RL

Robot Flag IDs:			
Numeric	Mnemonic	FS Reg Bit	Description
44	GA		Gripper Active
45	IA		Interface Active
46	RX		Radio Comm.
47	AS		Area Sensor
48	NB	b0	North/Front Sensor 1 (bumper)
49	EB	b1	East/Right Sensor 1 (bumper)
50	SB	b2	South/Back Sensor 1 (bumper)
51	WB	b3	West/Left Sensor 1 (bumper)
52	NS	b4	North/Front Sensor 2
53	ES	b5	East/Right Sensor 2
54	SS	b6	South/Back Sensor 2
55	WS	b7	West/Left Sensor 2
56	NT	b8	North/Front controller 1 (thruster)
57	ET	b9	East/Right controller 1 (thruster)
58	ST	b10	South/Back controller 1 (thruster)
59	WT	b11	West/Left controller 1 (thruster)
60	NC	b12	North/Front controller 2
61	EC	b13	East/Right controller 2
62	SC	b14	South/Back controller 2
63	WC	b15	West/Left controller 2

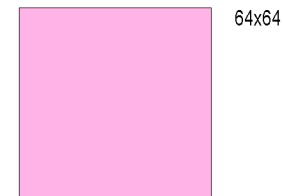
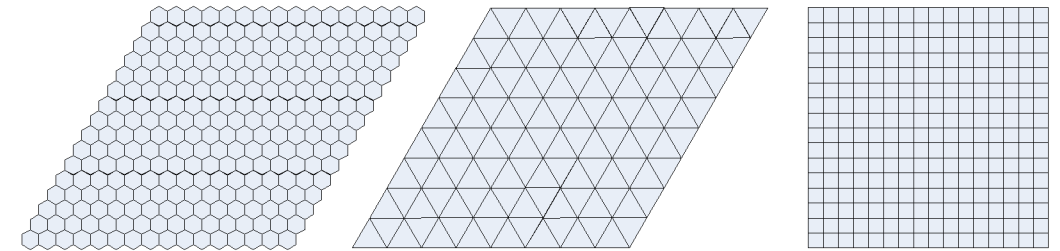
Special hybrid opcode/conditional examples:

```
PCD Relative right if north bumper is activated:
NB,ON,RR = COND NB opcode + CST,RR conditional
```

PCD Relative left if gripper is not grabbing anything:
GA,OFF,RL = COND GA opcode + CSF,RL conditional

Terminate process if interface disconnects:
IA,OFF,TRM = COND IA opcode + CSF,TRM conditional

COND opcodes are only active for the current instruction word, and they override any FSEL selection during the current instruction word execution.



Robot programming area recursive fill example:

[illegible]